

we don't need to. We won't be emitting any signals ourselves, but we do have a private slot, since we'll be subscribing to the `newConnection()` signal that will be emitted by the `QTcpServer` every time there's a new incoming connection.

Now for the headers; QtCreator will have automatically put a `#include <QObject>` statement into the file, but we'll need a few more headers. Here are all the headers we will need:

```
#include <QObject>
#include <QTcpServer>
#include <QTcpSocket>
#include <QHostAddress>
#include <QList>
#include <QByteArray>
```

That's right! In Qt5, every component has its own header, so you'll need to include every one of them manually.

You'll also need to go ahead and instruct `qmake` to enable the networking libraries of Qt; so open up the *FortuneServer.pro* file (it'll be in the *Project* pane, on the top-left corner of the screen), and near the top, add the following line...

```
QT += network
```

...so that it looks something like what's shown below:

```
QT    += core
QT    += network
QT    -= gui
```

Hit `Ctrl+S` to save the file. You'll see a bunch of small progress bars zip by on the bottom-right corner of the screen, as QtCreator takes into account the additional libraries we just enabled and rebuilds the code-completion databases for this project.

We can now start laying down some actual code. Open the *fortuneserver.cpp* file. By default, it'll include its own header file (*fortuneserver.h*) and it'll also have an empty body for the constructor function.

Let's fill up that constructor:

```
FortuneServer::FortuneServer(QObject *parent) :
    QObject(parent)
{
    // first we set up the server and make it listen
    // for new connections on 127.0.0.1:56789

    server = new QTcpServer(this);
    if (!server->listen(QHostAddress::LocalHost, 56789)) {
        qFatal("ERROR: Failed to bind TCP server to port
56789 on 127.0.0.1");
    }

    // now we connect the server's new connection signal
```

```
// to the new connection slot in this object
```

```
connect(server, SIGNAL(newConnection()), this,
        SLOT(sendFortune()));
```

```
// let's populate the fortune list
```

```
fortunes.append(QByteArray("You've been leading a dog's
life. Stay off the furniture.\n"));
fortunes.append(QByteArray("You've got to think about
tomorrow.\n"));
fortunes.append(QByteArray("You will be surprised by a loud
noise.\n"));
fortunes.append(QByteArray("You will feel hungry again in
another hour.\n"));
fortunes.append(QByteArray("You might have mail.\n"));
fortunes.append(QByteArray("You cannot kill time without
injuring eternity.\n"));
fortunes.append(QByteArray("Computers are not intelligent.
They only think they are.\n"));
```

```
// and we're done here
}
```

Again, the code should be pretty self-explanatory and the comments should help, but I will mention a few things here and there.

We start by creating a new `QTcpServer` object, and making it bind to `QHostAddress::LocalHost` (which is just an alias for 127.0.0.1) and port 56789. It's a high port so we won't need root privileges to bind to it. If you want to make an IPv6 server, you can just use the following code: `QHostAddress::LocalHostIPv6`.

You'll also notice that the code calls `qFatal()` with a message if the `listen()` fails. `qFatal()` prints the message to `stderr` and then immediately crashes the program. It doesn't clean anything up, and lets the operating system deal with it. On Linux, this isn't a problem, but on other platforms you should do a little housekeeping before you call `qFatal()`.

The next line of code is something you should get familiar with. This is how you connect a signal on some object to a slot. The syntax for the function is:

```
connect(
    pointer_to_object_that_emits_signal,
    SIGNAL(signal_name(argument_type, argument_type)),
    pointer_to_object_whose_slot_i_want_to_connect_to,
    SLOT(slot_name(argument_type, argument_type))
);
```

It's important that you wrap the signal in the `SIGNAL` macro and the slot in the `SLOT` macro. Also, do not mention any argument names, just the types. You actually emit a signal like this:

```
emit mySignal(myData);
```